ARCHITECTURE GUIDE

Linux USB gadget mode as a process boundary

ConfigFS, FunctionFS, the UDC, descriptor-driven endpoint discovery, file-descriptor capabilities, and disposable device workers.

01 The supervisor constructs and binds the device.

02 The worker receives open USB resources, never privileged paths.

03 The kernel carries payloads directly between host and worker.

04 Worker exit is the complete reconnect and reset primitive.

Offline companion to docs/usb-gadget-architecture.md

01 / MENTAL MODEL

The four layers

Layer	What it is	Responsibility
UDC	USB Device Controller hardware + Linux driver	Moves electrical USB traffic and exposes one bindable controller name.
ConfigFS	Kernel configuration filesystem	Builds the host-visible gadget: identity, configurations, function links, and UDC binding.
FunctionFS	Kernel API exposed as files	Accepts interface descriptors on `ep0`, creates endpoint files, and reports runtime events.
Worker	Unprivileged device implementation	Handles CTAP, CCID, Trezor messages, crypto, policy, display, buttons, and state.

The resulting stack

```
host application / OS class driver
| USB packets
v
physical link -> UDC -> composite gadget
|
FunctionFS / HID FDs
|
v
unprivileged worker
```

ConfigFS describes and assembles the device. FunctionFS supplies user-space interfaces. The UDC makes the assembled gadget electrically present to the host.

Binding is the last privileged action

The gadget remains invisible while it is being constructed. Writing the selected UDC name to the gadget's ConfigFS `UDC` attribute is the logical equivalent of plugging in the cable. Writing an empty value disconnects it.

02 / CONSTRUCTION

Descriptors become capabilities

A device profile contains complete FunctionFS v2 descriptor and string blobs. The supervisor parses them before the worker exists. It validates their structure, derives endpoint order and direction, publishes the blobs to `ep0`, then opens exactly the files the kernel creates.

Descriptor fact	Supervisor consequence	Worker receives
One OUT endpoint	Open generated endpoint read-only	Readable endpoint FD
One IN endpoint	Open generated endpoint write-only	Writable endpoint FD
Interrupt IN endpoint	Preserve declaration order	Third endpoint FD in its fixed slot
Multiple speed sets	Require identical topology	One stable logical layout
FunctionFS strings	Validate table, then write to ep0	No string-file or mount path

Why the worker still keeps ep0

Publishing descriptors is finished before handoff, but `ep0` remains the FunctionFS control plane. The worker reads `BIND`, `ENABLE`, `DISABLE`, `UNBIND`, `SUSPEND`, `RESUME`, and class/vendor `SETUP` events there.

Other USB descriptors do not arrive later. The device, configuration, interface, endpoint, HID report, and FunctionFS string data must all exist before binding. Runtime responses are protocol data, not late descriptor publication.

The supervisor can verify that the profile's declared endpoint topology matches the exact FD bundle. This removes repeated descriptor-writing code from every worker and makes malformed profiles fail before USB attachment.

03 / FILE-DESCRIPTOR HANDOFF

A tiny resource protocol

The supervisor and worker share an `AF_UNIX/SOCK_SEQPACKET` socket placed on fixed worker descriptor 3. Each normal-data record is eight bytes; open file descriptions travel separately as `SCM_RIGHTS` ancillary data. The record declares the exact attached FD count.

```
0..3 magic: UGSP
4 version: 1
5 message type
6..7 exact FD count, big-endian
```

Direction	Message	Value	FDs
Supervisor -> worker	PREBIND_RESOURCES	0x01	FunctionFS ep0 and endpoints
Worker -> supervisor	PREPARED	0x81	0
Supervisor -> worker	POSTBIND_RESOURCES	0x02	ConfigFS HID nodes; explicit 0 is valid
Worker -> supervisor	SERVING	0x82	0

What can be transferred

Any descriptor that the receiving process can meaningfully use may be inherited or sent: regular files, pipes, terminals, device nodes, event descriptors, and Unix, TCP, or UDP sockets. The receiver gets a reference to the same open file description: file status flags and offsets are shared, while descriptor-table flags such as close-on-exec belong to each process's descriptor entry.

The sender cannot use `SCM_RIGHTS` to invent extra kernel permissions. Access mode was fixed when the file was opened, and the worker's later operations remain subject to that file's driver behavior and ordinary process security rules.

USB, I2C, SPI, and GPIO descriptors are sent together in fixed pre-bind message slots. The environment carries no descriptor numbers; it contains only the persistent and runtime directory paths.

04 / LIFECYCLE

One worker, one incarnation

```
PREPARING
create gadget; publish/open FunctionFS
| send pre-bind FDs
v
AWAITING PREPARED -> BINDING -> send post-bind FDs
|
v
AWAITING SERVING
|
v
SERVING
| worker exit / EOF
v
CLEANING: unbind, close, reap, unmount, remove
|
+----> new PREPARING
service stop: CLEANING -> supervisor exits
```

The two startup acknowledgements protect different boundaries. `PREPARED` means the worker has validated and initialized the pre-bind resources, so it is safe to expose the USB identity. `SERVING` means post-bind resources exist and the worker has accepted them.

HID creates the only required second handoff: ConfigFS cannot create `/dev/hidgN` until after UDC binding. A FunctionFS-only worker such as Virtual Trezor still receives an explicit zero-FD post-bind record, keeping one fixed startup state machine.

There is no light reconnect command. A firmware reconnect is worker exit. The supervisor stays alive, unbinds first, destroys every resource from that incarnation, and launches a fresh Unix process. A service stop performs the same cleanup and then ends the supervisor.

Process creation is the reset primitive: threads, buffers, endpoints, and capabilities cannot leak from one incarnation into the next.

05 / DEVICE WORKERS

The supervisor leaves the data plane

After startup, the kernel moves USB payloads directly between the host controller and worker-held FDs. The privileged process handles metadata and lifecycle only; it never sees CTAP frames, APDUs, Trezor protobuf messages, PINs, seeds, keys, or YubiHSM commands.

Device	Kernel surface	Worker traffic
Virtual YubiKey	ConfigFS HID + FunctionFS CCID	FIDO HID reports; CCID bulk OUT/IN, interrupt IN, and ep0 events
Virtual Trezor	FunctionFS vendor interface	64-byte interrupt OUT/IN packets and ep0 events
Virtual YubiHSM	FunctionFS vendor bulk interface	Bulk request/response frames and ep0 events

Virtual YubiKey

The pre-bind bundle is CCID `ep0`, bulk OUT, bulk IN, and interrupt IN. The post-bind bundle is the FIDO HID descriptor. The worker owns CCID framing, PC/SC semantics, CTAPHID, applets, credentials, and persistent authenticator state.

The profile advertises manufacturer `Virtual USB Gadget` and product `Virtual Yubico YubiKey FIDO+CCID`. Compatibility software may still display an inferred model name such as 'YubiKey 5A'; that UI label is not a USB manufacturer assertion and cannot necessarily be controlled by descriptor strings.

Virtual Trezor

The pre-bind bundle is main `ep0`, OUT, IN, display bus, and GPIO. The post-bind bundle is empty. Upstream firmware logic continues to compose the genuine 128x64 framebuffer. The Linux worker uses those received descriptors for the selected display and buttons, and clears the display on orderly exit.

Virtual YubiHSM

The same FunctionFS pattern supports a vendor bulk device. Its profile supplies the descriptors; its worker owns commands, sessions, objects, capabilities, audit behavior, and state. The supervisor needs no YubiHSM-specific code.

06 / CLIENT BEHAVIOR

What the host actually talks to

There is no extra network client or server. To macOS, Linux, or Windows the Raspberry Pi is an ordinary USB peripheral. Host class drivers claim interfaces and expose their normal APIs to applications.

USB interface	Host-side path	Typical application
HID / FIDO	OS HID + FIDO stack	Browser, WebAuthn client, libfido2
CCID	USB CCID driver + PC/SC	Yubico Authenticator, OpenSC, smart-card tools
Trezor vendor interface	Trezor transport library	trezorctl, Trezor Suite
Vendor bulk	libusb or product library	YubiHSM connector/client

Why FIDO can work while CCID is exclusive

HID does not inherently make a device shareable. FIDO and CCID remain independently usable because they are separate USB interfaces claimed by separate host-driver stacks. An application's exclusive PC/SC transaction on the CCID interface does not claim the HID interface.

The same distinction applies inside the worker: each interface has different framing and endpoint semantics even when both ultimately reach applets in one virtual device. HID is report-oriented and host-polled; CCID and vendor protocols use their declared bulk or interrupt endpoints.

The host never knows about ConfigFS, FunctionFS, SCM_RIGHTS, or worker processes. Those are Linux implementation details behind the USB identity it enumerates.

07 / UDC DISCOVERY

Pi 4 and Pi 5 use the same model

USB gadget mode depends on a USB Device Controller, not on I2C target/peripheral support. Raspberry Pi 4 and Pi 5 both use Linux DWC2 device mode on the gadget-capable USB-C connection. The Pi 5's ordinary RP1 USB host ports are not extra gadget controllers.

```
/sys/class/udc/
fe980000.usb # typical Pi 4 name
1000480000.usb # typical Pi 5 name

supervisor: sort names -> choose first
override: --udc EXACT_NAME
```

When can more than one UDC exist?

- A test system loads dummy_hcd and exposes one or more software controllers.
- A virtualized or emulated platform supplies several device controllers.
- Custom carrier hardware exposes an additional physical device-mode controller.
- A non-Pi system genuinely contains multiple gadget-capable controllers.

One selected UDC exposes one profile at a time. Virtual YubiKey and Virtual Trezor can both be installed, but two simultaneously enumerated identities require two independent controllers or two Pis.

Power and the host port

A Pi 5 can draw substantial power compared with a small USB token. Gadget-mode software does not protect against an inadequate cable, supply, or host-port power budget. A separately powered Pi may still use the data connection, but power topology must avoid unintended back-power paths and follow the board's electrical guidance.

UDC names are discovered from sysfs because they are kernel/hardware identifiers, not stable product names to hard-code in a worker.

08 / OPERATIONS AND TRUST

The installed contract

Artifact	Owner	Current contract
Profile	Device project; root-installed	Schema 1; USB identity; descriptor blobs; worker and hardware resources
Supervisor	Privileged service	Validate, construct, open, transfer, bind, monitor, unbind, clean
Worker protocol	Matched deployment set	UGSP version 1; fixed 8-byte records and fixed resource order
Worker	Unprivileged device project	Protocol logic, secrets, state, UI, and direct endpoint I/O

Capability boundary

The worker receives open resources, not permission to explore privileged namespaces. FunctionFS remains root-owned. The control socket is fixed at FD 3; every other handle arrives through `SCM_RIGHTS`. The cleared environment contains only the persistent and runtime directory paths, while configured worker options remain command arguments.

This is useful privilege separation, not hardware isolation. A software authenticator or wallet on a general-purpose Pi remains vulnerable to compromise of its worker account, kernel, storage, memory, supply chain, and physical environment.

Operational checks

- Validate the profile before installation and keep the installed copy root-owned.
- Confirm endpoint count, order, direction, and host-visible interface order.
- Observe PREPARED, UDC bind, FunctionFS ENABLE, SERVING, and teardown logs.
- Kill the worker and verify unbind-first cleanup followed by a fresh process.
- Stop the service and verify final UDC, ConfigFS, mount, display, and process cleanup.

The design is deliberately literal: a virtual firmware implementation reads and writes real kernel endpoint file descriptors. The supervisor supplies those capabilities and controls when the host can see them.

Detailed, browser-readable documentation: docs/usb-gadget-architecture.md